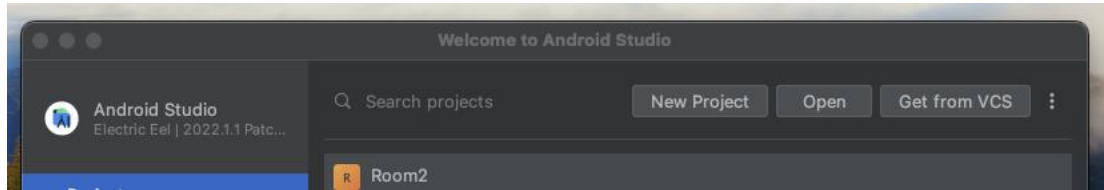
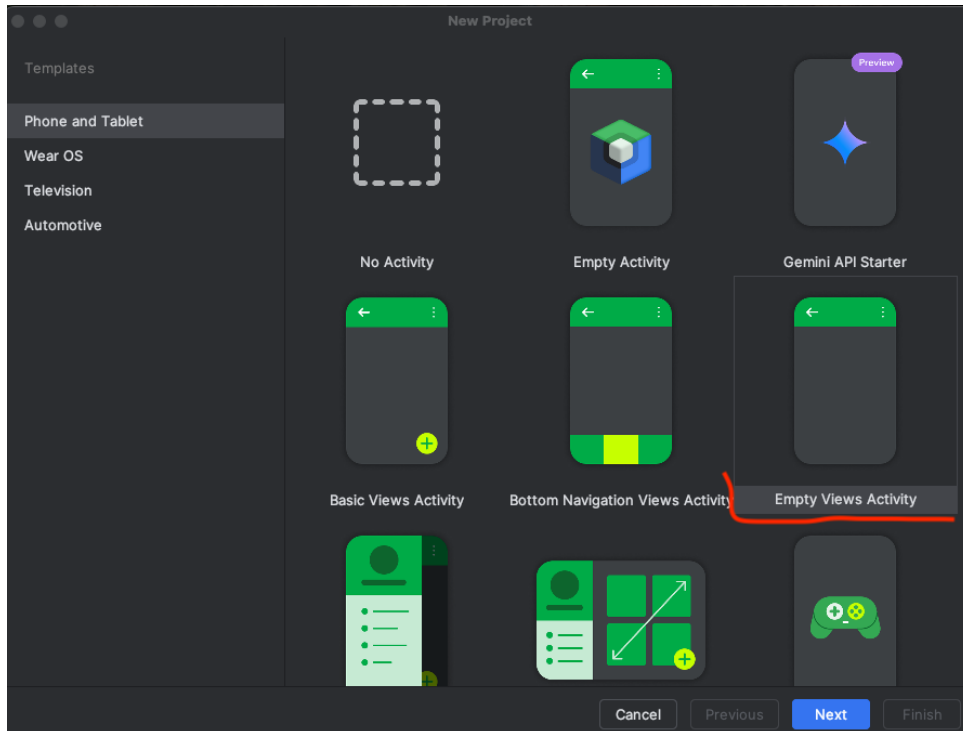


Android Apps – Static Layouts

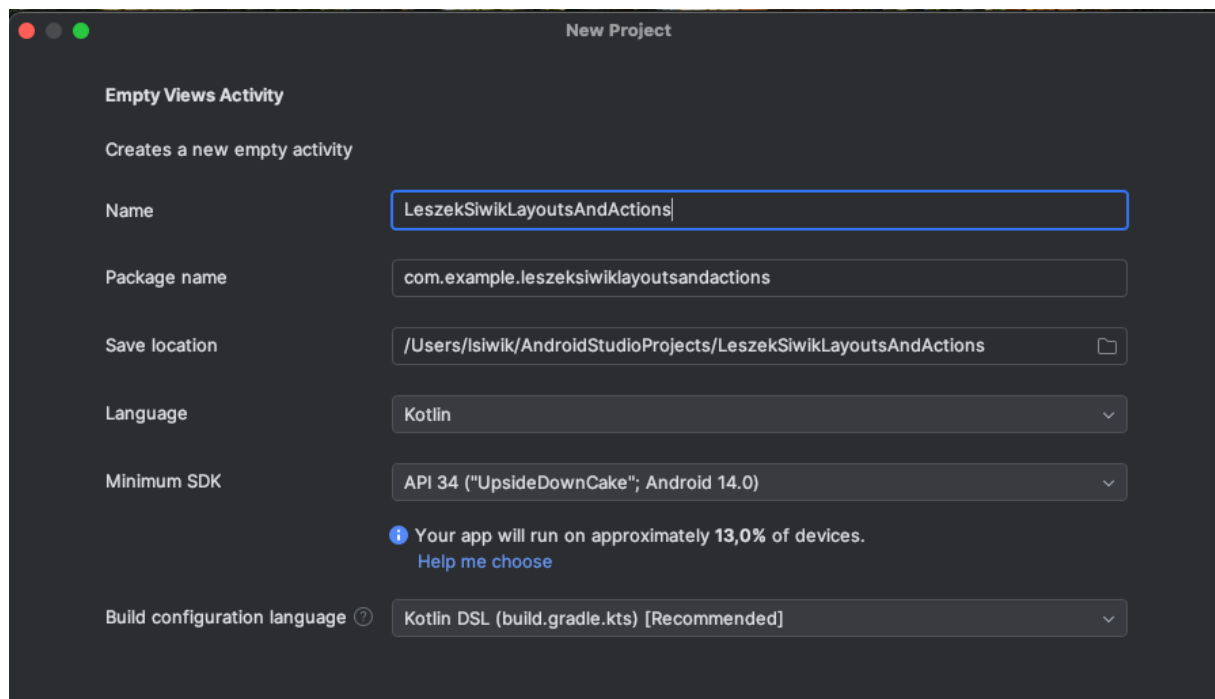
- Launch Android Studio
- Create a new project (File -> New Project or New Project from a Startup window)



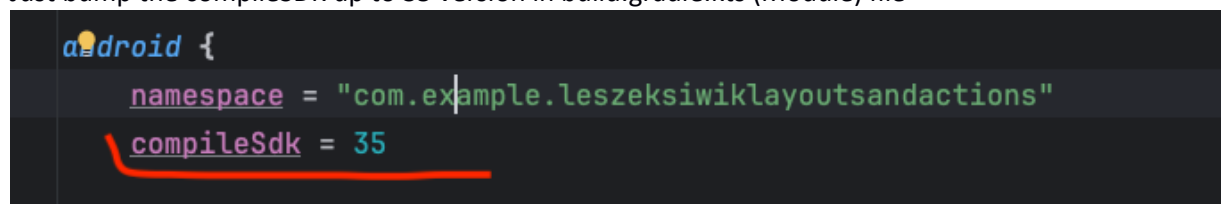
- Select Phone and Tablet and Empty Views Activity as a project template



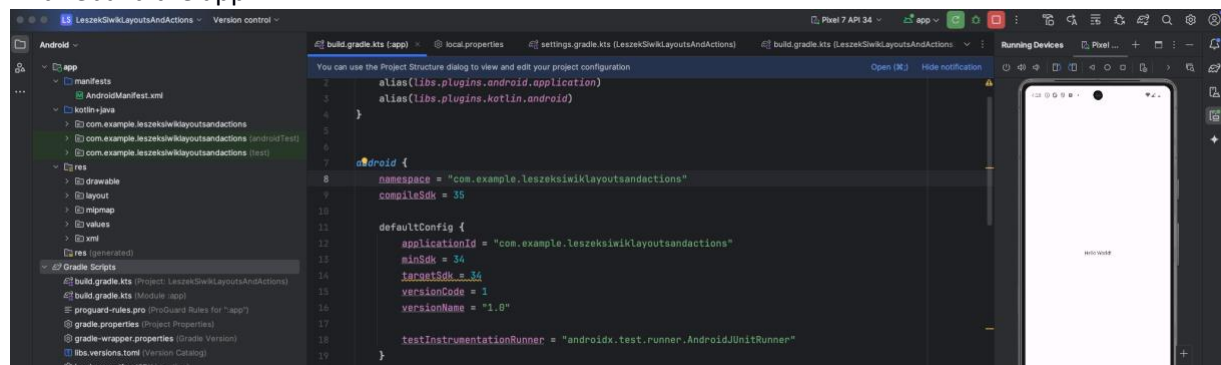
- On the next screen set:
 - FirstnameSurnameLayoutsAndActions as the project name
 - Default value as the package name and project location
 - Kotlin as project language
 - API 34 Android 13.0 (UpsideDownCake) as the Minimum SDK
 - And press Finish



- Wait until Gradle finishes his job and let's launch the app to make sure that everything is fine. If you are getting an error saying that Dependency 'androidx.core:core:1.15.0' requires libraries and applications that depend on it to compile against version 35 or later of the Android APIs. Just bump the compileSdk up to 35 version in build.gradle.kts (Module) file



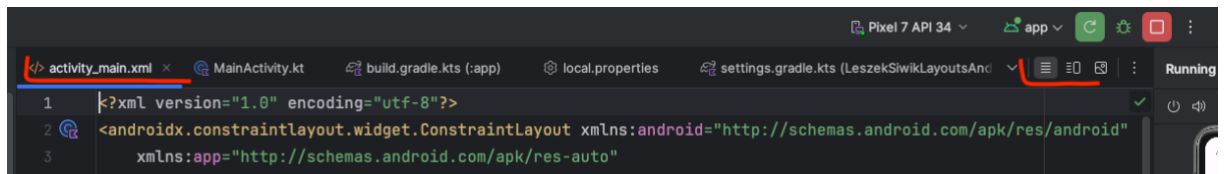
- And rebuild the app.



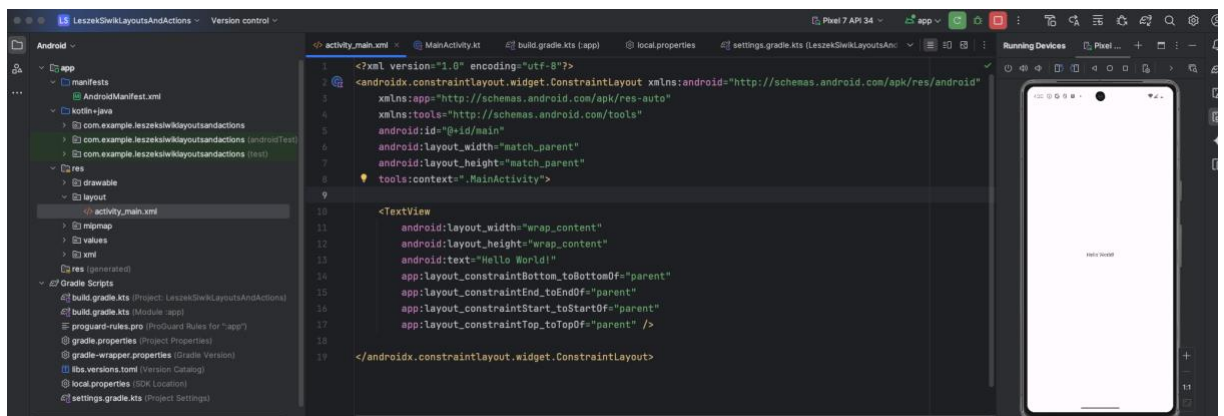
• Working with static Layouts

- Definition of how our first activity (first screen) looks like is contained in the res -> layout -> activity_main.xml file

- Because we are practicing, please complete the ongoing exercises by editing the xml file(s) directly!
- Possibly the most convenient view mode for us will be a split view showing both: the UI source code and the designer where you may follow the effects of your changes made in the source code. So please change the view mode into the SplitView



- And you should get something like:

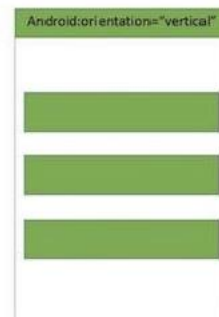
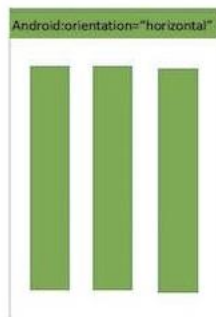


• Layouts

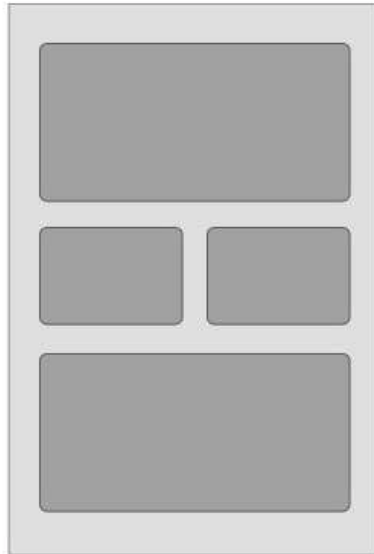
- When we are defining the user interface, we may use “static” layouts - a type of containers, defined in the xml files, where we may drop off user interface elements like buttons, lists, text fields, other nested layouts, etc.)
- Android SDK provides some predefined layouts like:

Sr.No	Layout & Description
1	Linear Layout LinearLayout is a view group that aligns all children in a single direction, vertically or horizontally.
2	Relative Layout RelativeLayout is a view group that displays child views in relative positions.
3	Table Layout TableLayout is a view that groups views into rows and columns.
4	Absolute Layout AbsoluteLayout enables you to specify the exact location of its children.
5	Frame Layout The FrameLayout is a placeholder on screen that you can use to display a single view.
6	List View ListView is a view group that displays a list of scrollable items.
7	Grid View GridView is a ViewGroup that displays items in a two-dimensional, scrollable grid.

- (https://www.tutorialspoint.com/android/android_user_interface_layouts.htm)
- LinearLayout – aligns all the elements dropped off inside linearly (vertically or horizontally depending on the “orientation”) one by one:

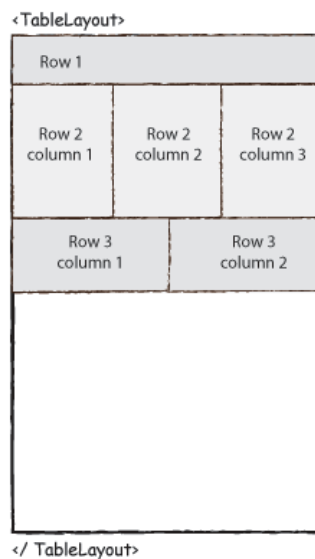


- More information about LinearLayouts can be found e.g. here: https://www.tutorialspoint.com/android/android_linear_layout.htm
- RelativeLayout - where elements are placed "relatively" to each other - e.g. we place something to the right of something, something else under something, etc.

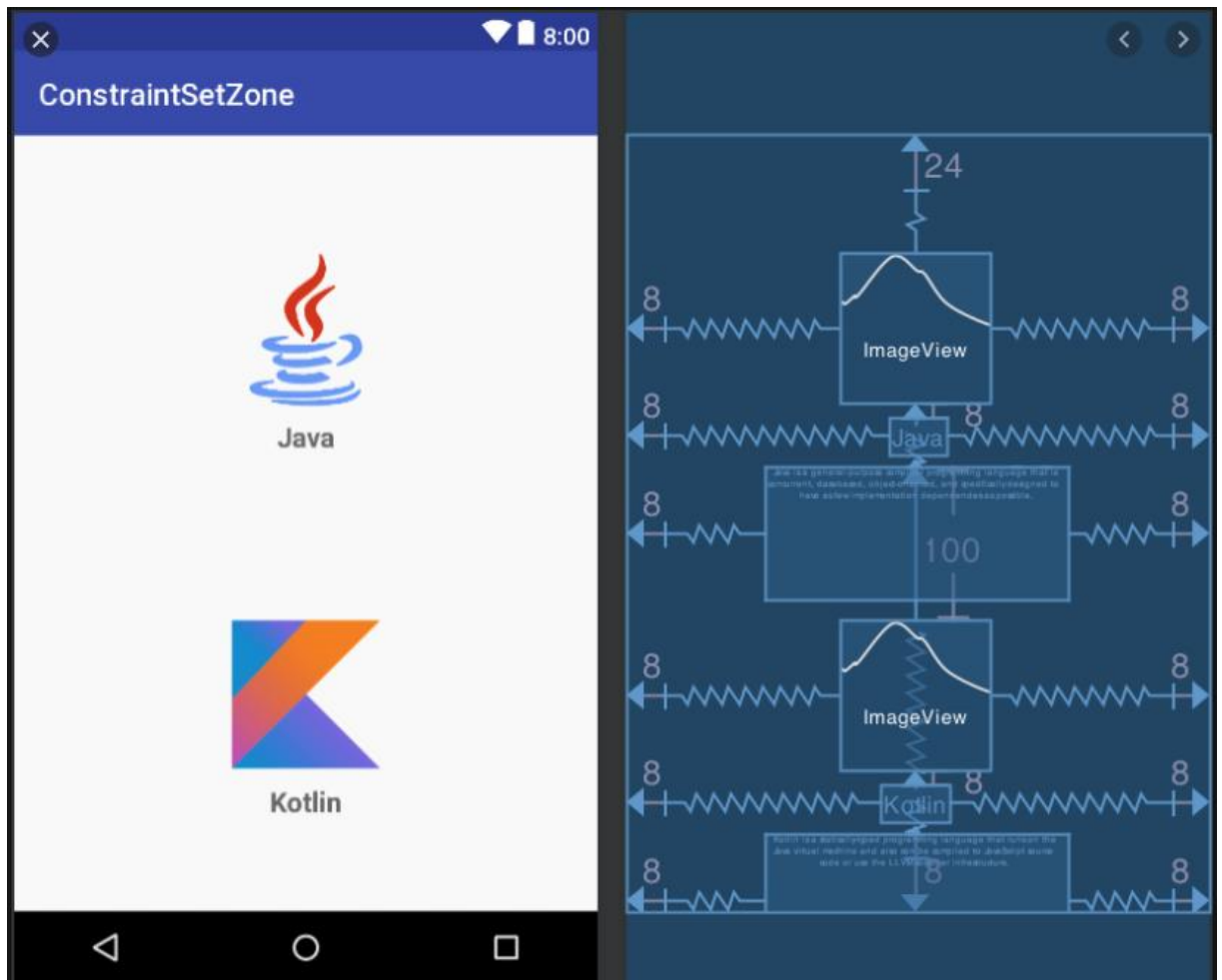


Relative Layout

- More information about RelativeLayouts can be found e.g. here: https://www.tutorialspoint.com/android/android_relative_layout.htm
- TableLayout – where elements are located in rows and columns

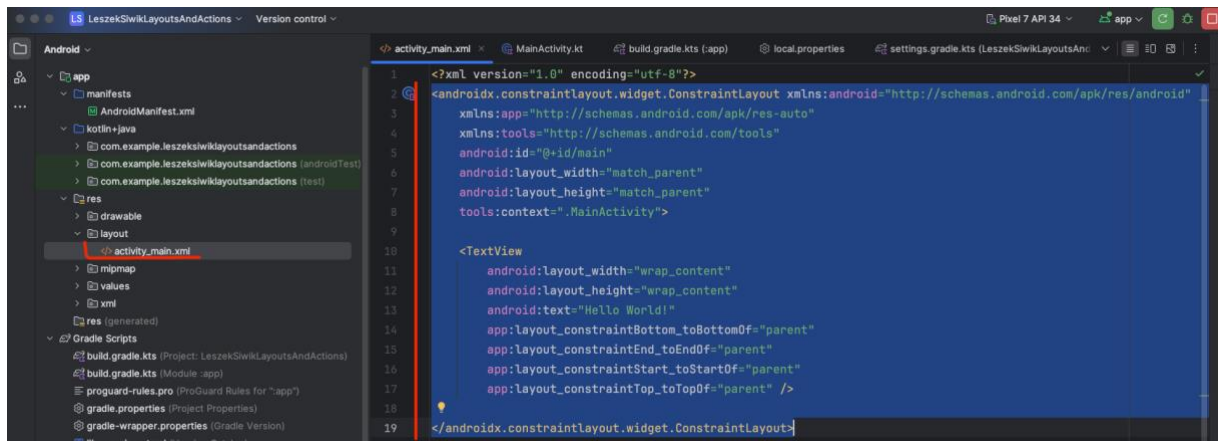


- More information about TableLayouts can be found e.g. here: https://www.tutorialspoint.com/android/android_table_layout.htm
- ConstraintLayout – where elements are "pinned together" with constraints/strings

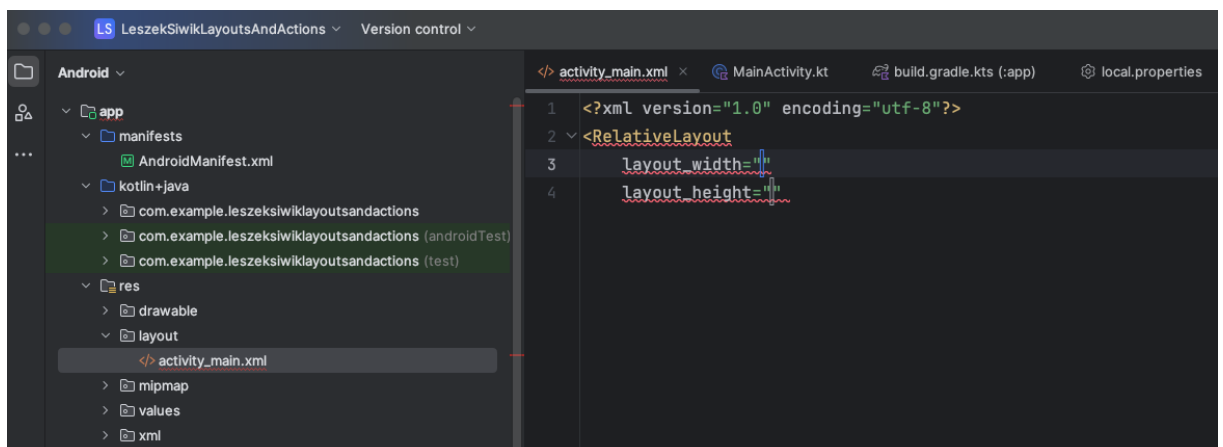
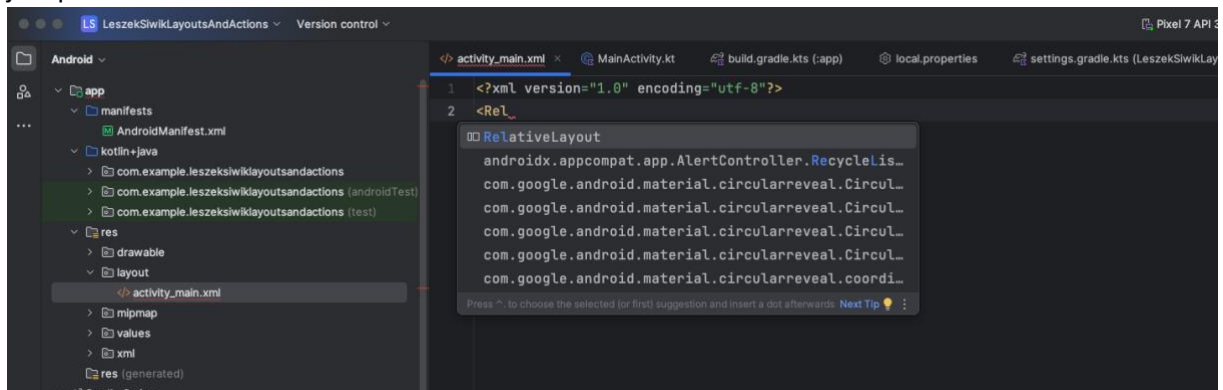


• RelativeLayout

- For the beginning, we start working with the RelativeLayout
- As it was said, the RelativeLayout is a "container" where elements are placed "relatively" to each other – i.e, we place something to the right of something, something else under something etc. We will work with it for a while now.
- Please, open the main_activity.xml file. Delete the ConstraintLayout definition added there by the wizard. (So remove lines 2 to 19)



- Instead, please start typing "<Rel" and when the intellisense shows up the RelativeLayout just press the enter:

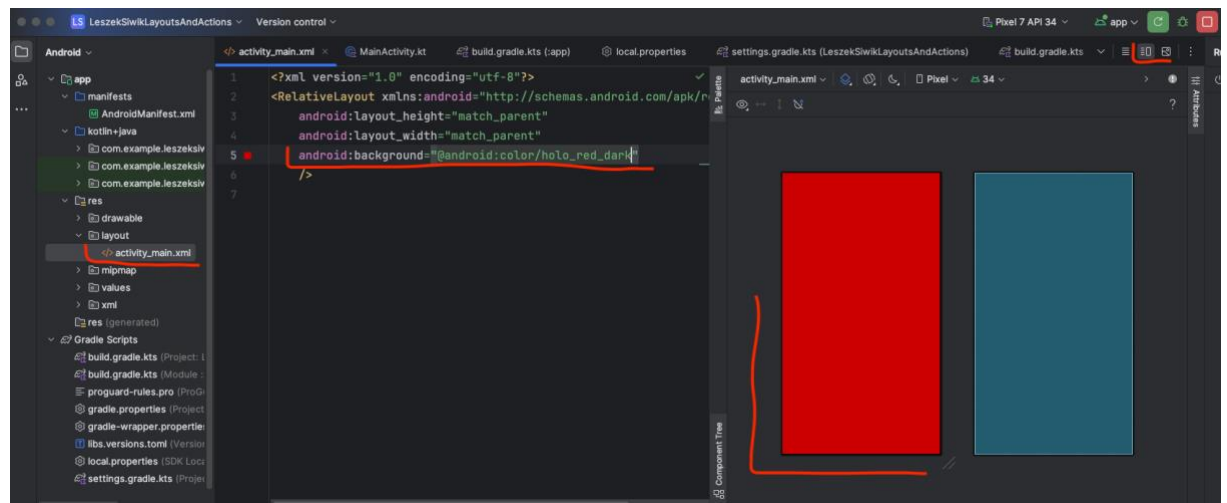


- As we can see, the RelativeLayout has two mandatory parameters - layout_width and layout_height. We set both to match_parent - which means that we want to define the layout as width and as height as its "parent" is, and because the created layout is the "top most" layout – (it is not nested in any other layout) - its "parent" is just the screen of the phone on which the application will be launched - so by setting layout_width and layout_height to "match_parent" we say that we want the created layout to be as wide and as high as the screen of the device on which the application will be launched .



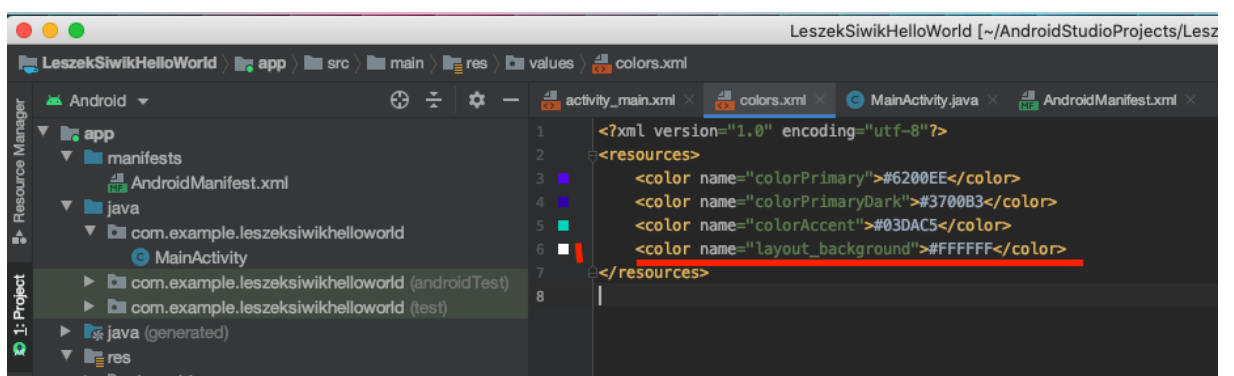
```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_height="match_parent"
4     android:layout_width="match_parent" />
5
```

- Additionally, let's set the background color of the layout, to red (or any other), what makes it easy to "track" of how the layout is defined. So, being inside the layout definition, we start typing "back" and Intellisense should support us:

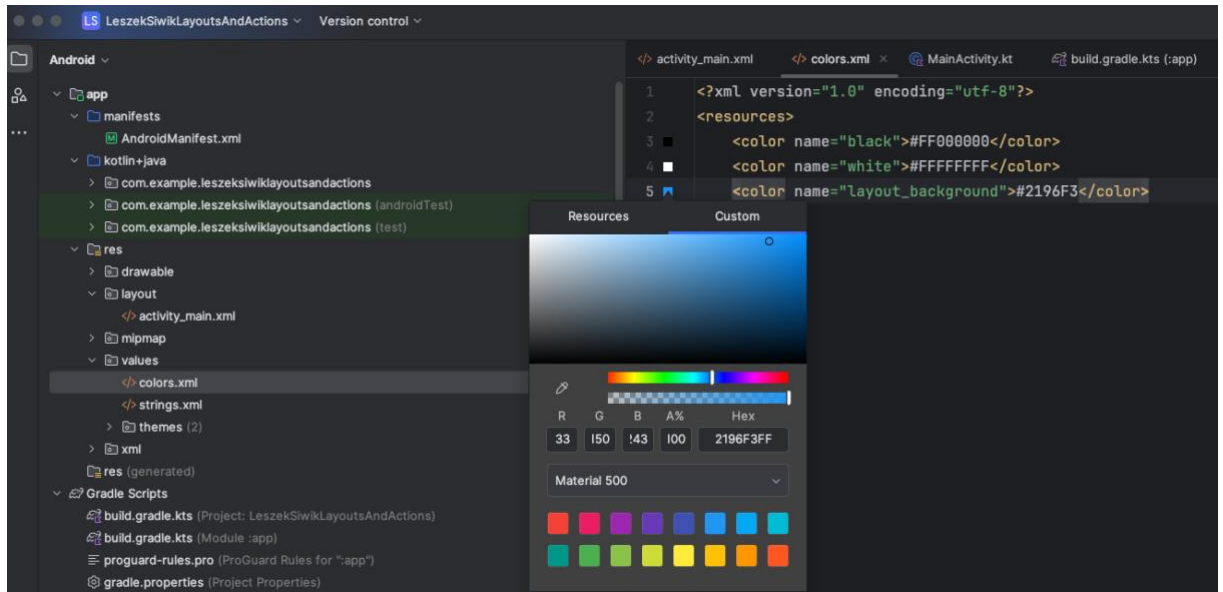


• Working with custom colors

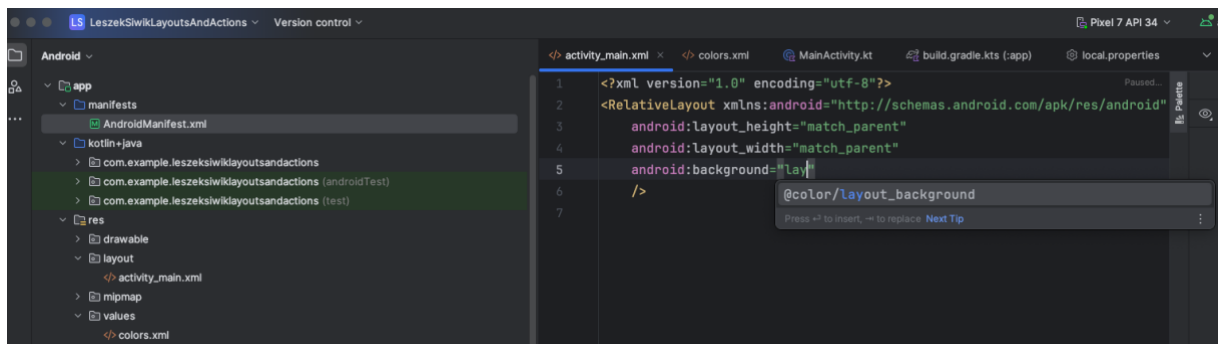
- As we touched the color topic - let's define our own color and set it as the layout background color. For this purpose:
 - Open res/values/colors.xml file
- Similarly to those already defined there - let's add a new "color item"
- Set the name of the color to be added (e.g "layout_background") and its value to #FFFFFF



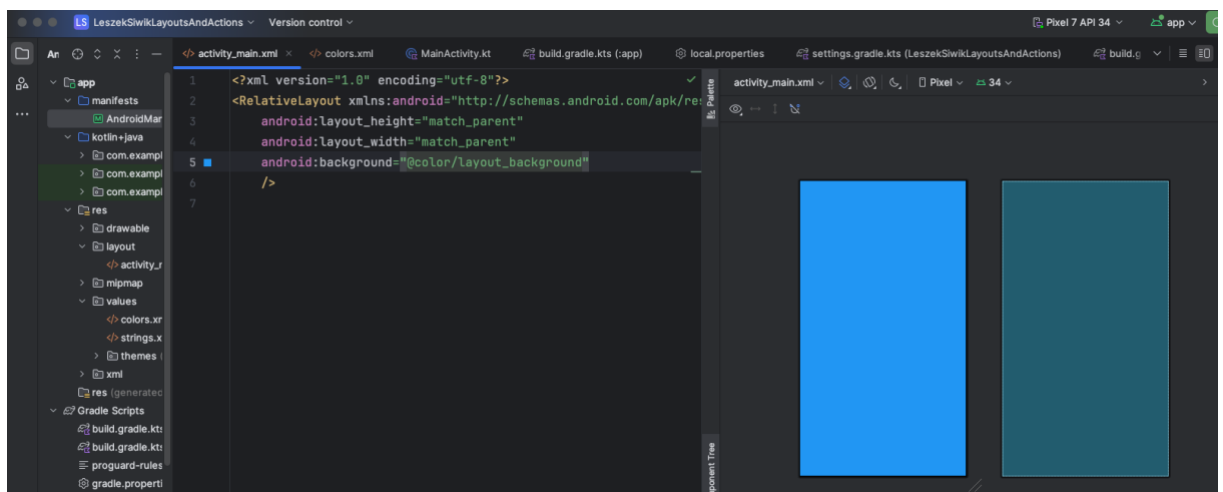
- Click on the white square to the right of the line number (for me to the right of line 6) and choose any color you want to be used



- Now, let's go back to activity_main.xml file and set the value of the layout_background property to the color we have just defined.

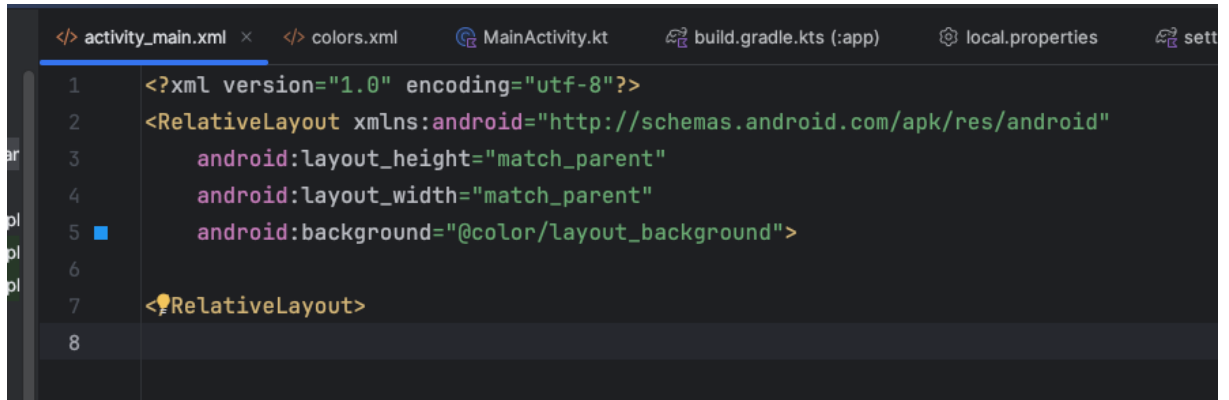


- The color change should be visible on the designer / preview



- **Working with label**

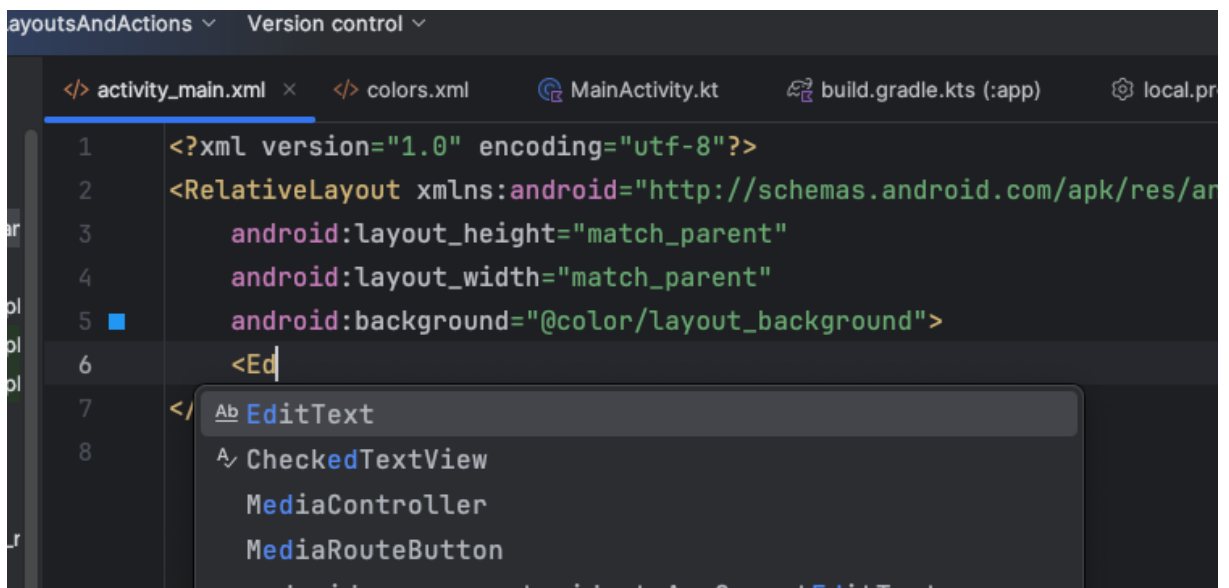
- Let's add some "label" (static inscription) to the layout, namely:
- Being in the activity_main.xml file - we split RelativeLayout definition into two separate tags - opening and closing ones.
- So, we change its definitions of our layout as follows:



```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_height="match_parent"
    android:layout_width="match_parent"
    android:background="@color/layout_background">

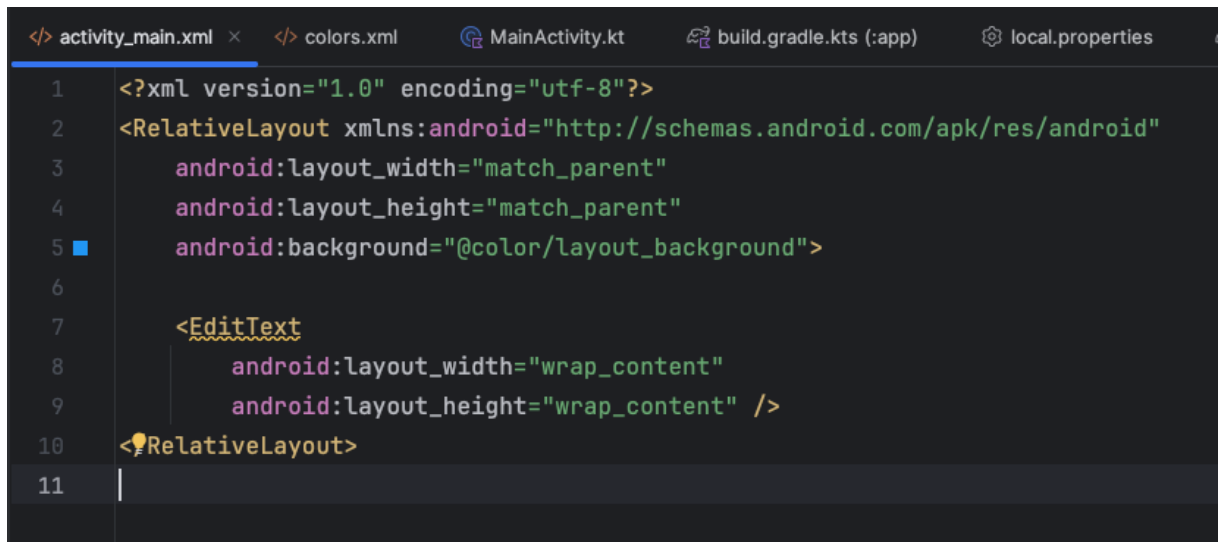
</RelativeLayout>
```

- And now, we may add inside the layout the EditText



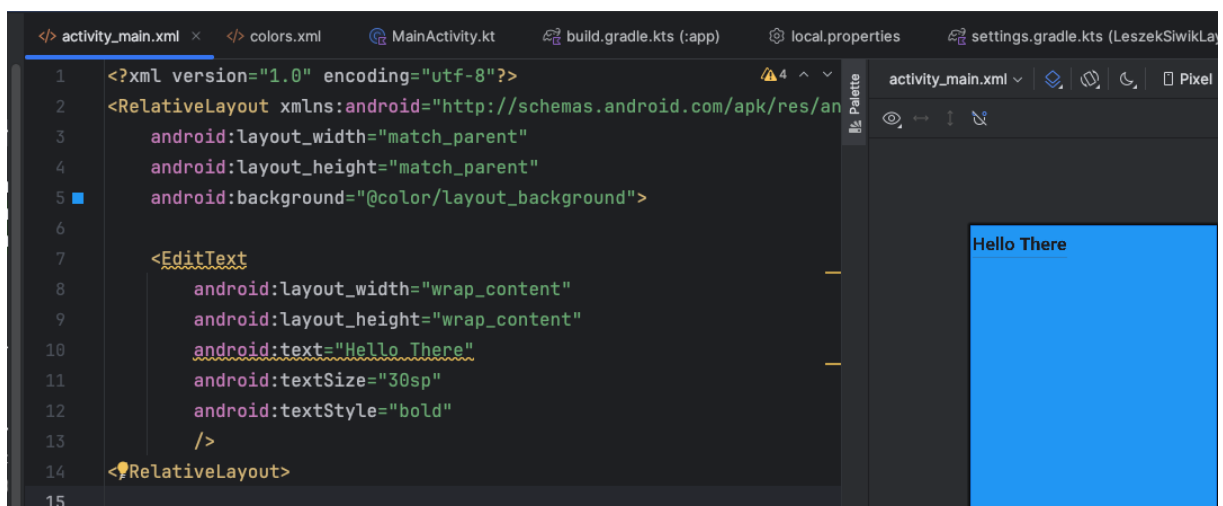
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/ar
    android:layout_height="match_parent"
    android:layout_width="match_parent"
    android:background="@color/layout_background">
<Ed
</
Ab EditText
A CheckedTextView
MediaController
MediaRouteButton
android.support.design.widget.AppCompatEditText
```

- This time, let's set the width and height of this element to "wrap_content" - that is, we want the EditText to be as wide and as high as the inscription we will place inside.



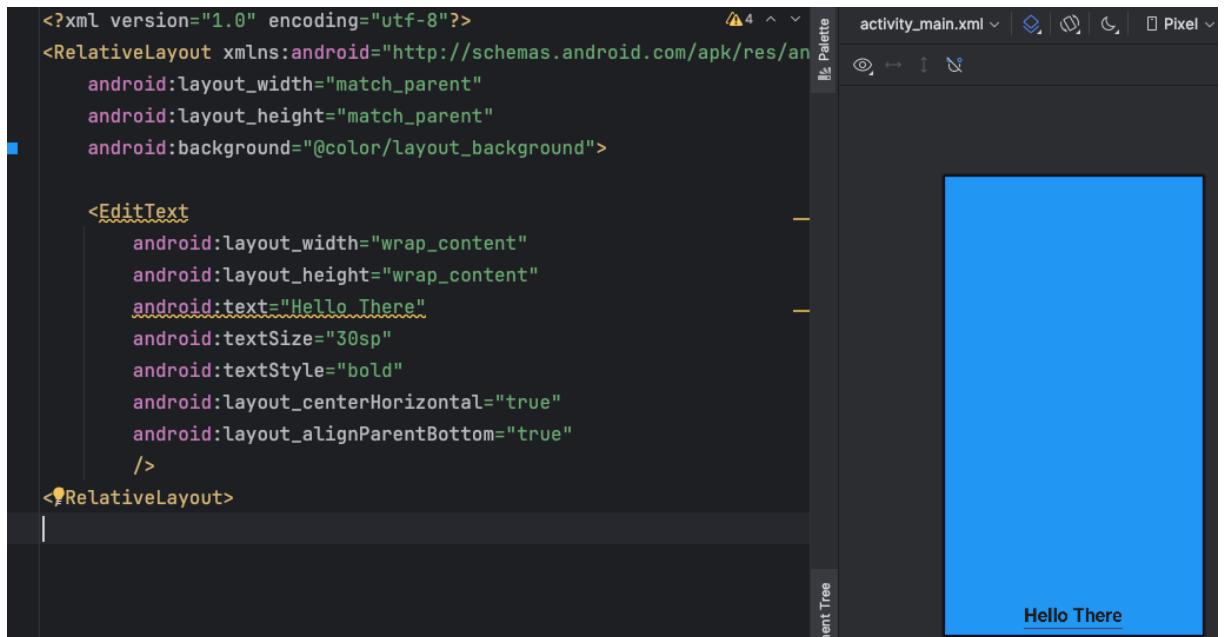
```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:background="@color/layout_background">
6
7     <EditText
8         android:layout_width="wrap_content"
9         android:layout_height="wrap_content" />
10 </RelativeLayout>
11
```

-
- Next, let's set the text property of our EditText to "Hello There",
- Additionally, let's set textSize to "30sp" and textStyle to Bold. The definition and effect should be as below:

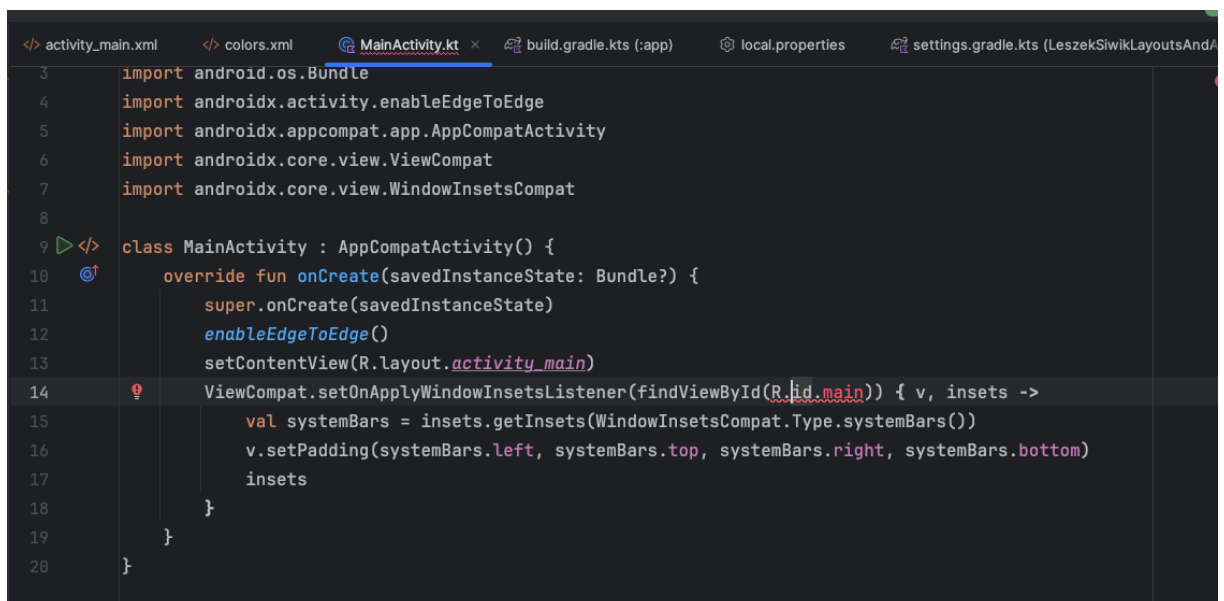


```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/an
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:background="@color/layout_background">
6
7     <EditText
8         android:layout_width="wrap_content"
9         android:layout_height="wrap_content"
10        android:text="Hello There"
11        android:textSize="30sp"
12        android:textStyle="bold"
13    />
14 </RelativeLayout>
15
```

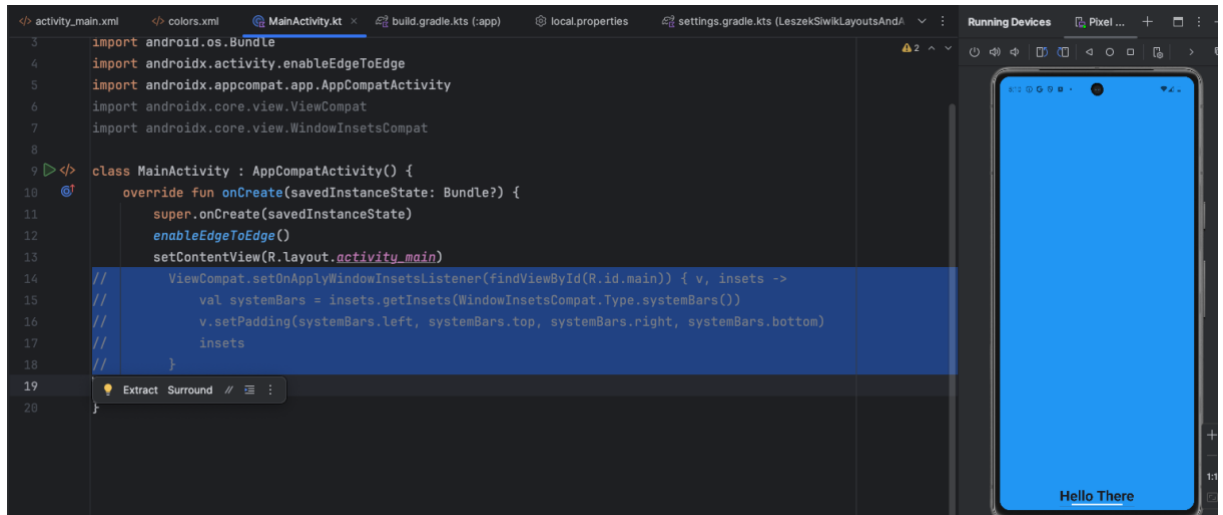
-
- The "sp" unit that we specified is so-called scale-independent-pixels, which means the abstract pixel size taking into account the physical "density" of the screen of the device on which the application is launched and user preferences (fonts size) as well. More information can be found, e.g. here: <https://stackoverflow.com/questions/2025282/what-is-the-difference-between-px-dip-dp-and-sp>
- Let's make our label to be centered horizontally (property: android:layout_centerHorizontal = "true") and let's place it at the bottom of the screen (property: android:layout_alignParentBottom = "true"). So the definition and the effect should be as follows:



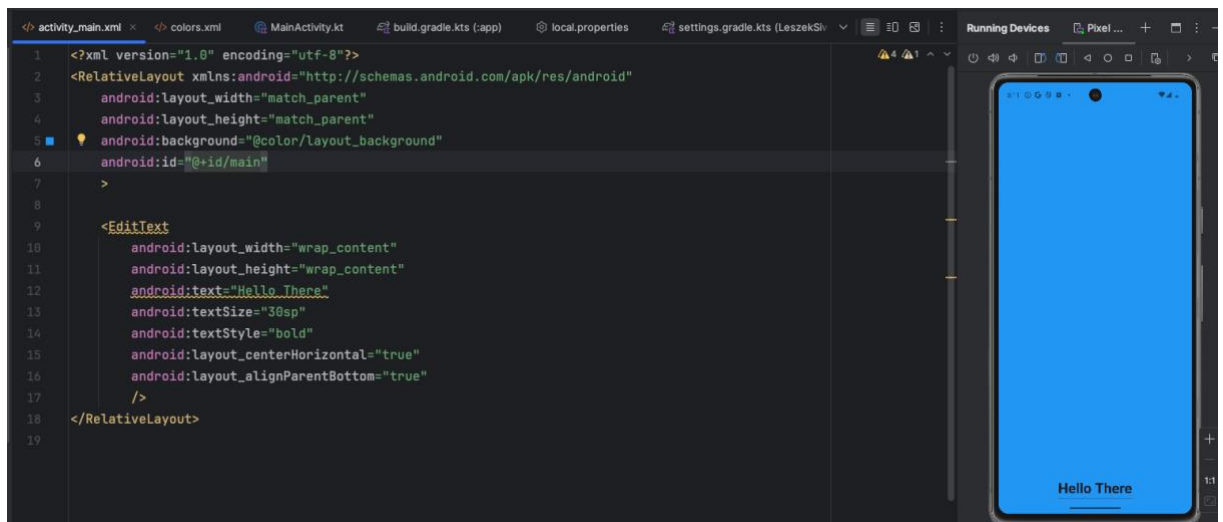
- Let's re-build and re-run the app to make sure that everything is fine. If you are getting the error saying that R.id.main is not recognized in MainActivity.kt



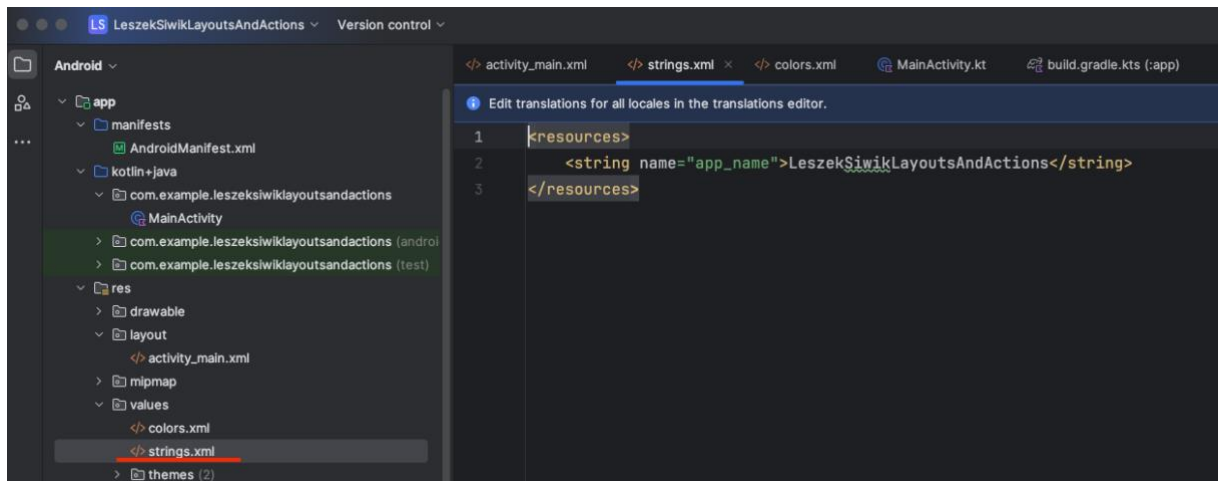
- You may either remove / comment out this insets part:



- Or generate/set the id to our layout like this:



- The text appearing in our EditText has been “hardcoded” in the definition of this label. Generally, when creating an Android application the convention is that strings are defined rather as application resources in the res -> values -> strings.xml file:



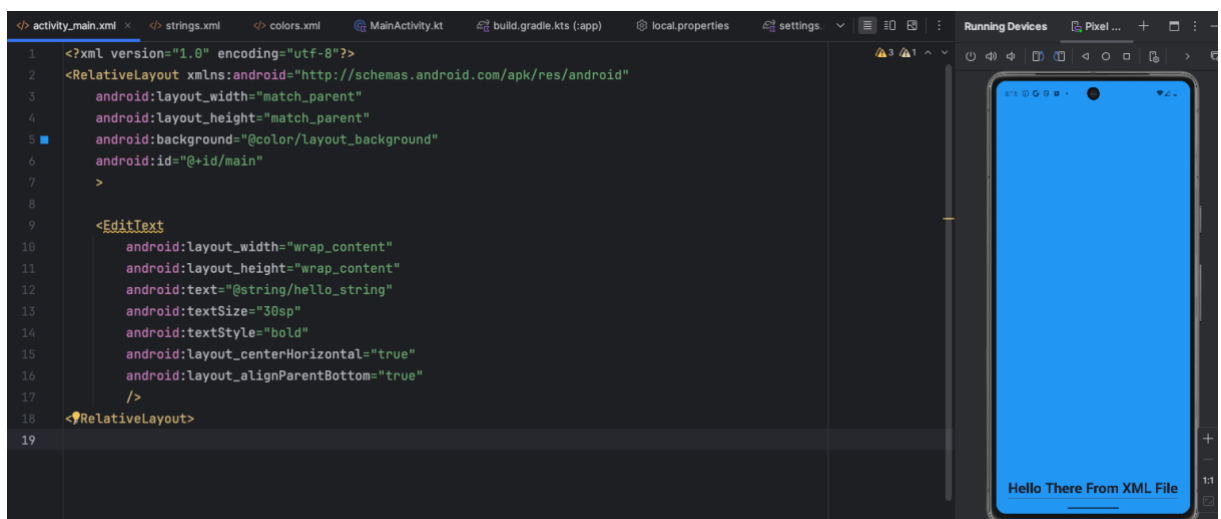
- This is to make the application maintenance easier (if we want to change some strings, or facilitate translating the application into various languages etc)
- So, to define our inscription as the app resource let's add a new "string item in stings.xml file.
- Let it be called hello_string and let's set its value to "Hello There from XML file"

```

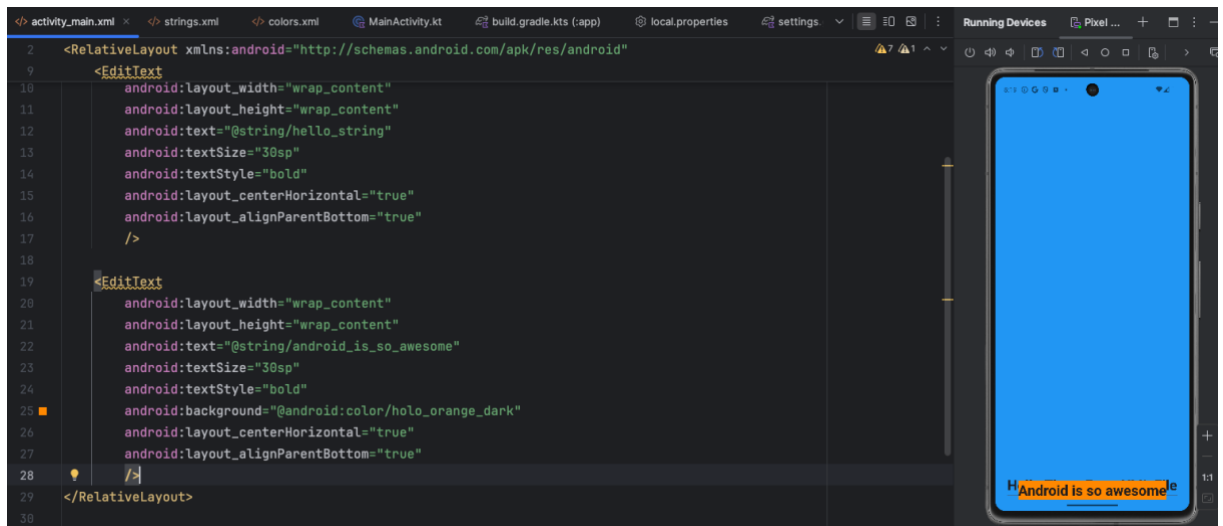
1  <resources>
2      <string name="app_name">LeszekSiwikLayoutsAndActions</string>
3      <string name="hello_string">Hello There From XML File</string>
4  </resources>

```

- Now, let's go back to activity_main.xml and let's change the value for the text property from the hardcoded value to @string/hello_string. So the definition and effect should be as below:



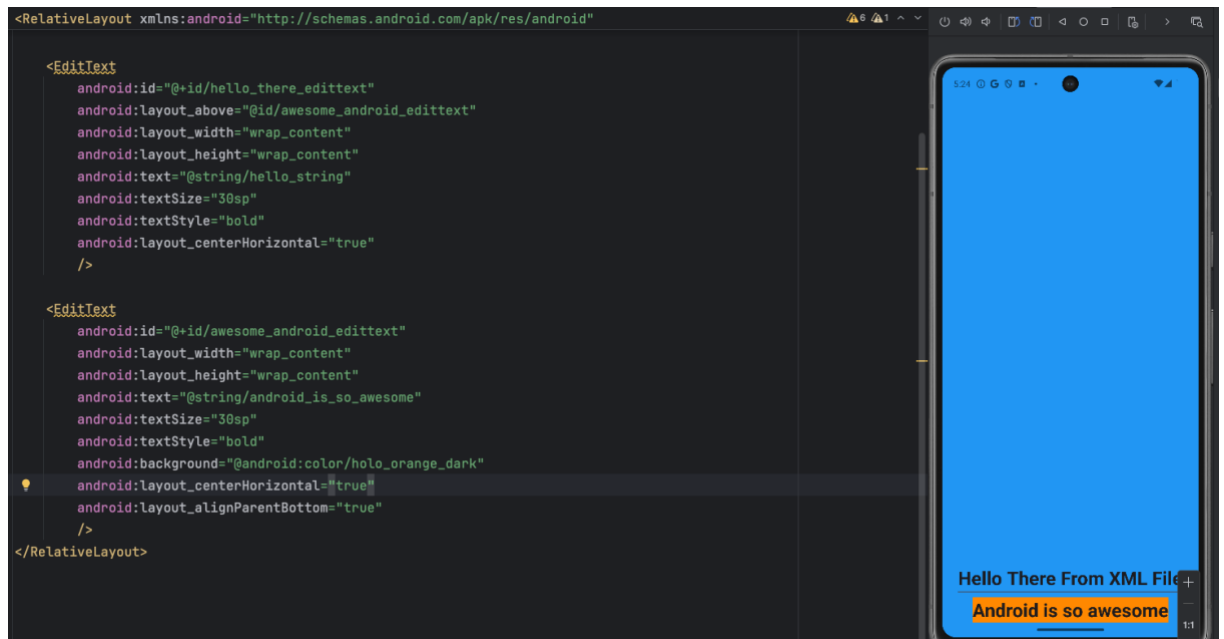
- Let's us now add a second EditText to the layout. Let's set the text inside the new label to "Android is so awesome". Additionally, let's set the background color to holo_orange_dark. We set the rest of the properties as in the previous case
- So now, without any further modifications, the effect will be like



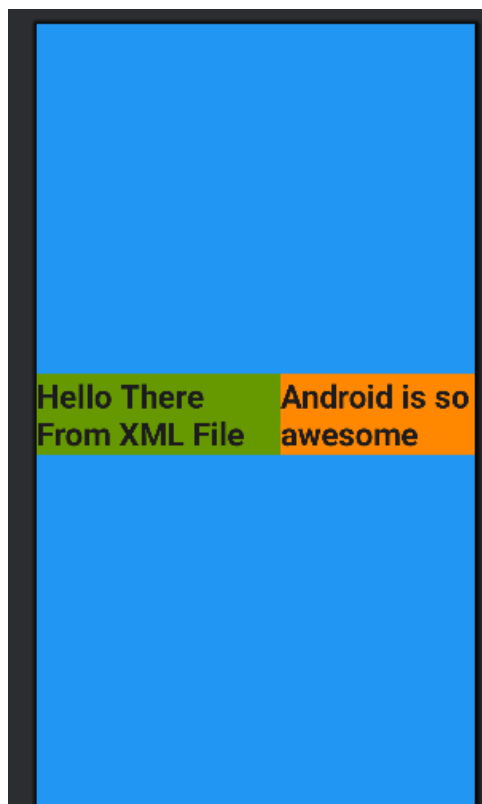
- So, the labels overlap since both are aligned to the bottom of the "parent"
- Let's fix it now and try to put the labels one under/above the other
- So, first, let's add "ids" to both edit texts like



- And now we may put the one above / below the other, e.g. like:



- For practicing, please place both EditTexts in the layout as below:

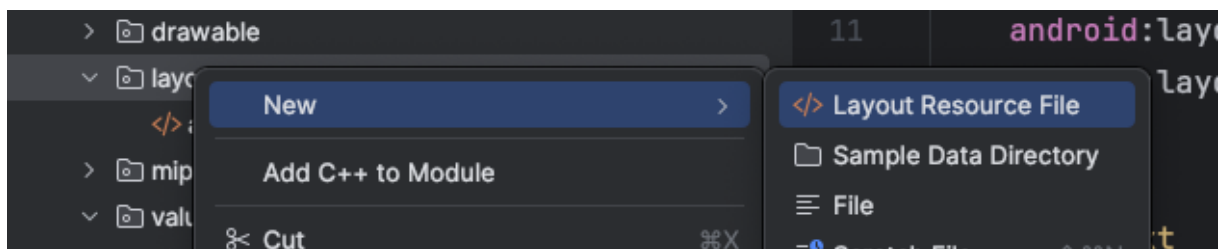


- **Working with new layout and (image) assets/resources**

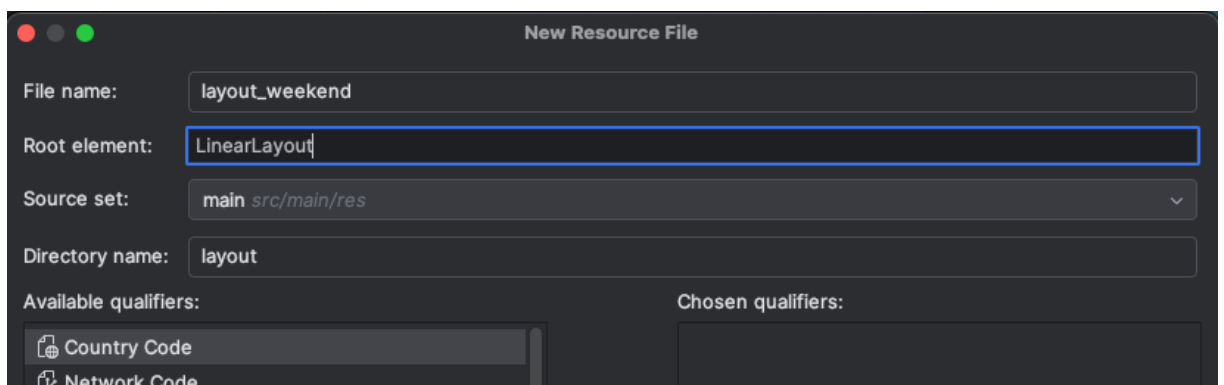
- Now, let's play with new resources, and let's prepare the new layout like the one below.



- But we will do it in a new/separate layout file. So, right click on the layout folder and then choose New -> Layout Resource File

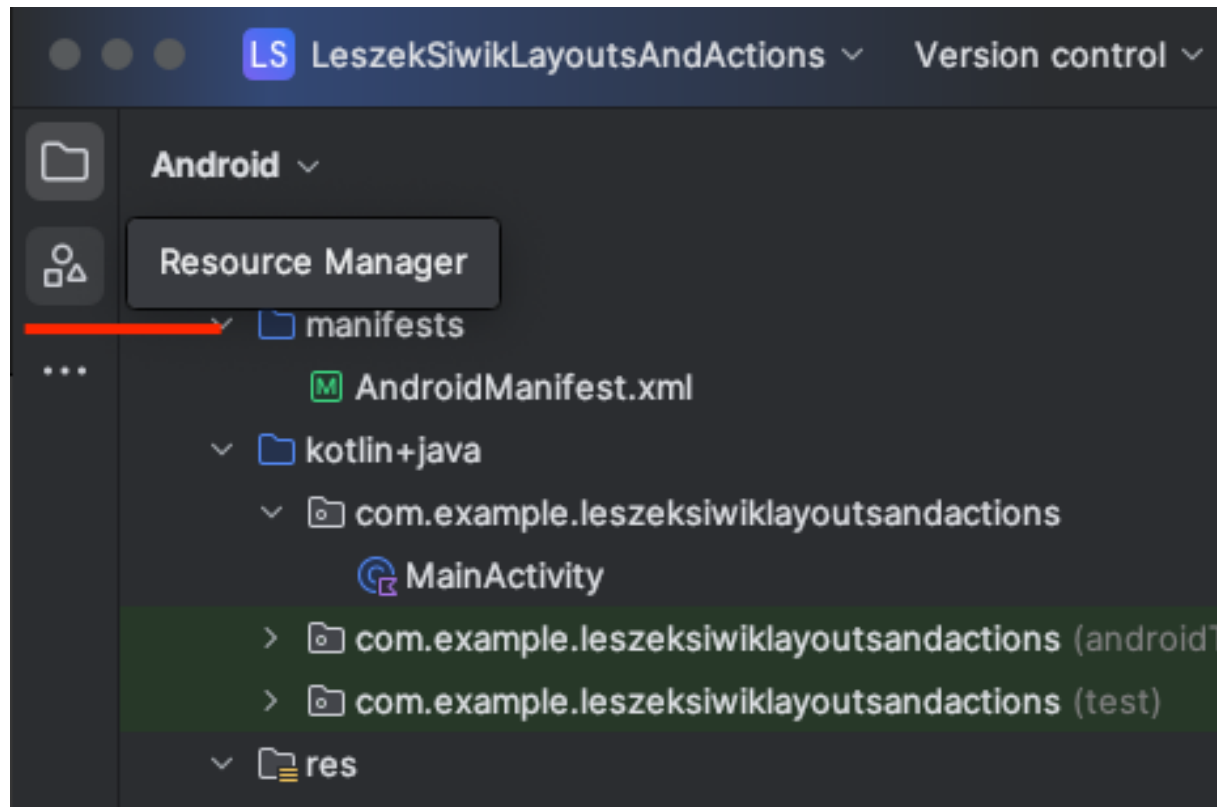


- In the wizard, set the file name to layout_weekend and the root element to LinearLayout

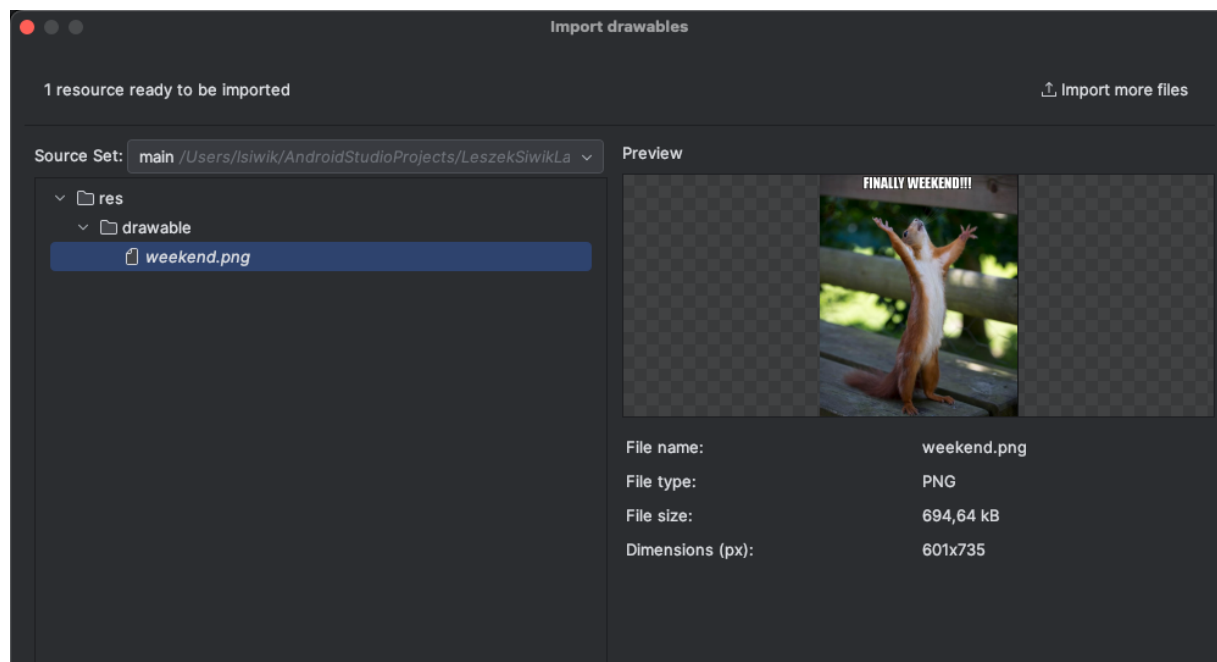


- Now, let's compose the layout out of two elements - EditText and ImageView.
- To set the ImageView content:

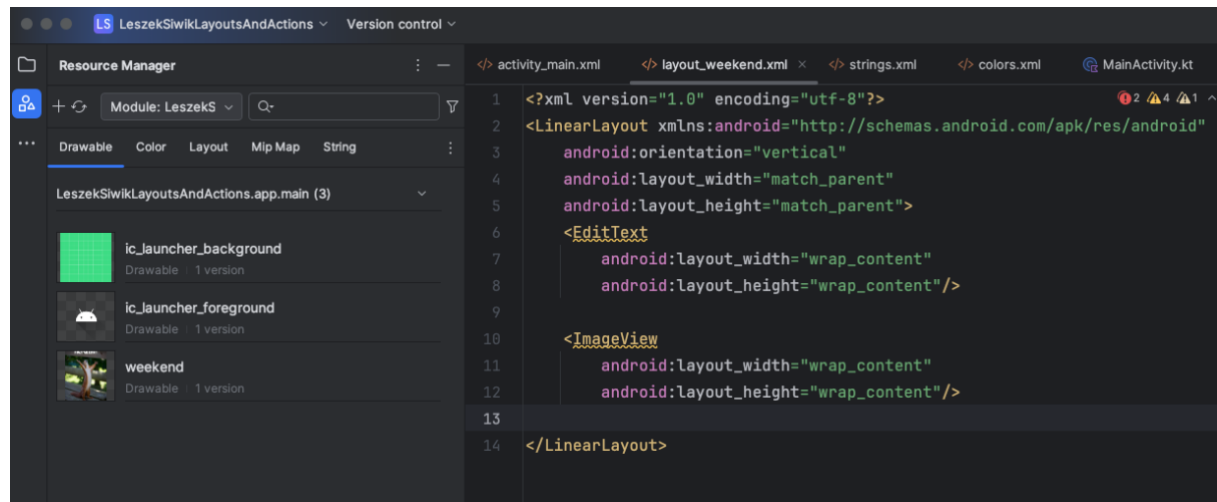
- If you want to use the same picture as I did – you can use the following file:
<https://drive.google.com/file/d/1THpELo2nZM1JGQYVey06hR5diBz-cOKI/view?usp=sharing>
- We need to add a picture to the project - so let's go to Resource Manager, Click on “+” and select “Import Drawables” option



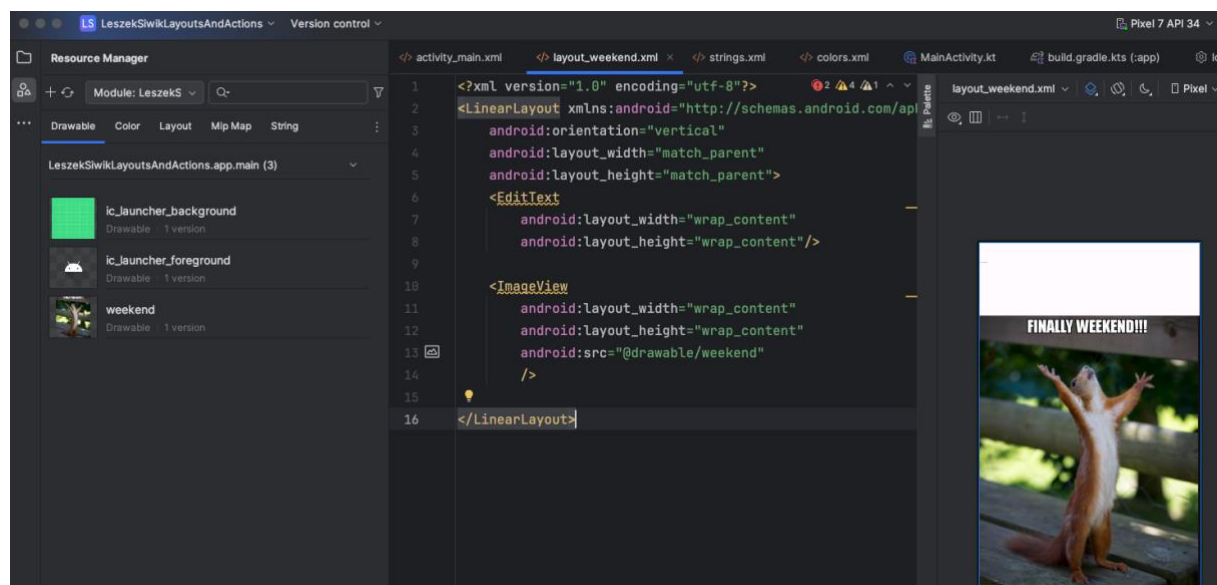
- Lets choose the file we want to import ,Click ok, then Next



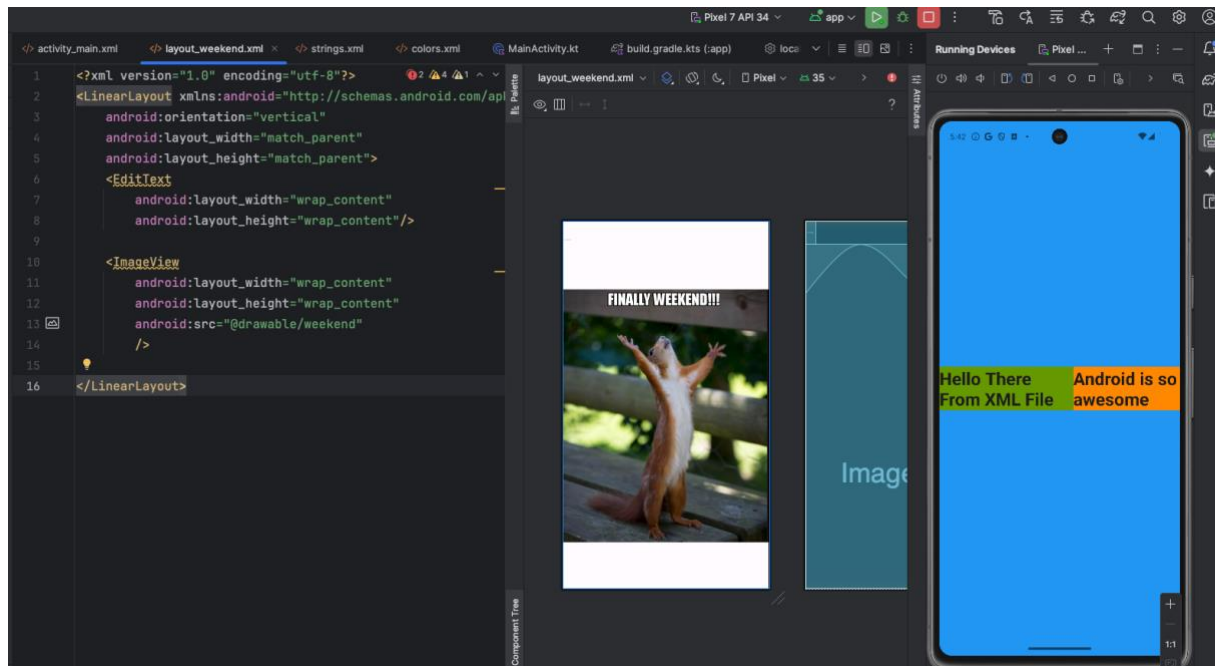
- And finally Import, and now the file should be listed as the Drawable in the Resource Manager



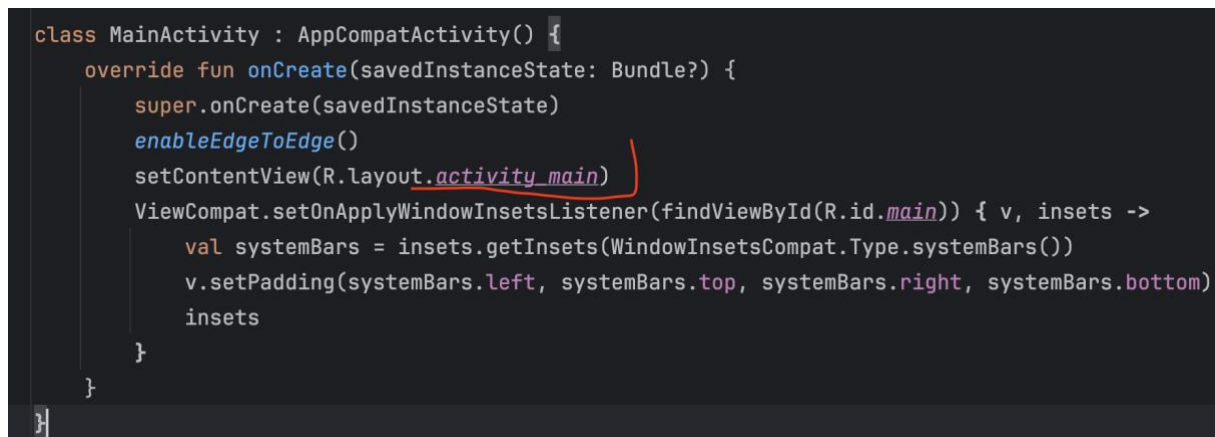
- Now, let's add to our new layout the ImageView element, let's set width to match_parent, height to wrap_content, src to @drawable/weekend and we should see our image added to the layout



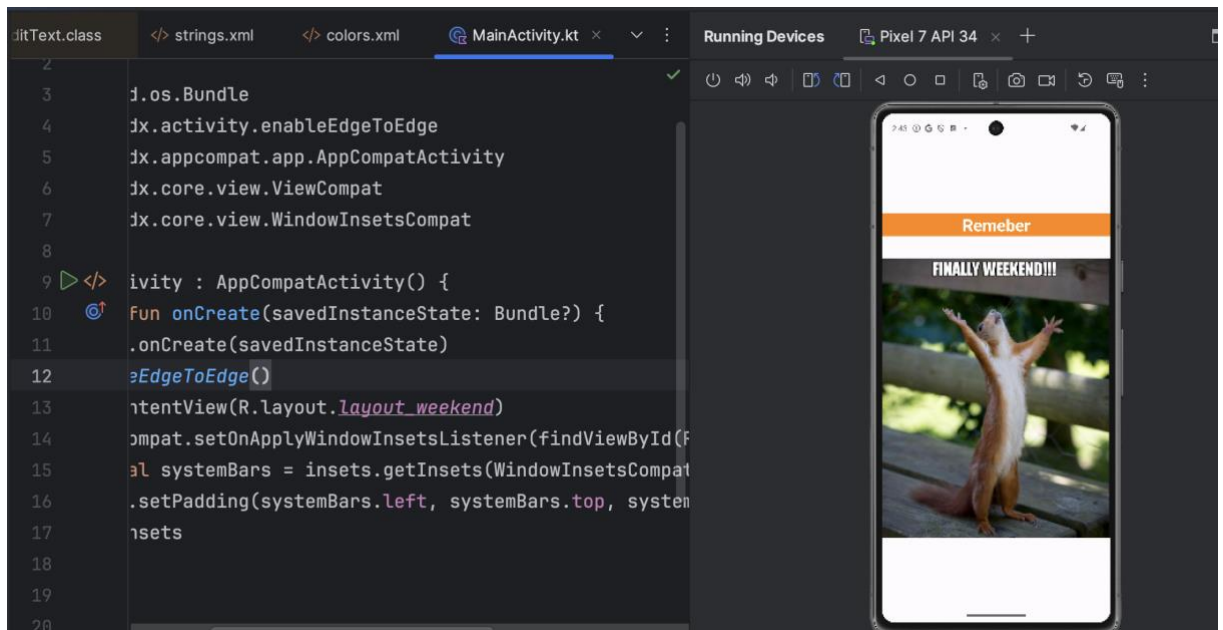
- The problem is that when we run the app we see:



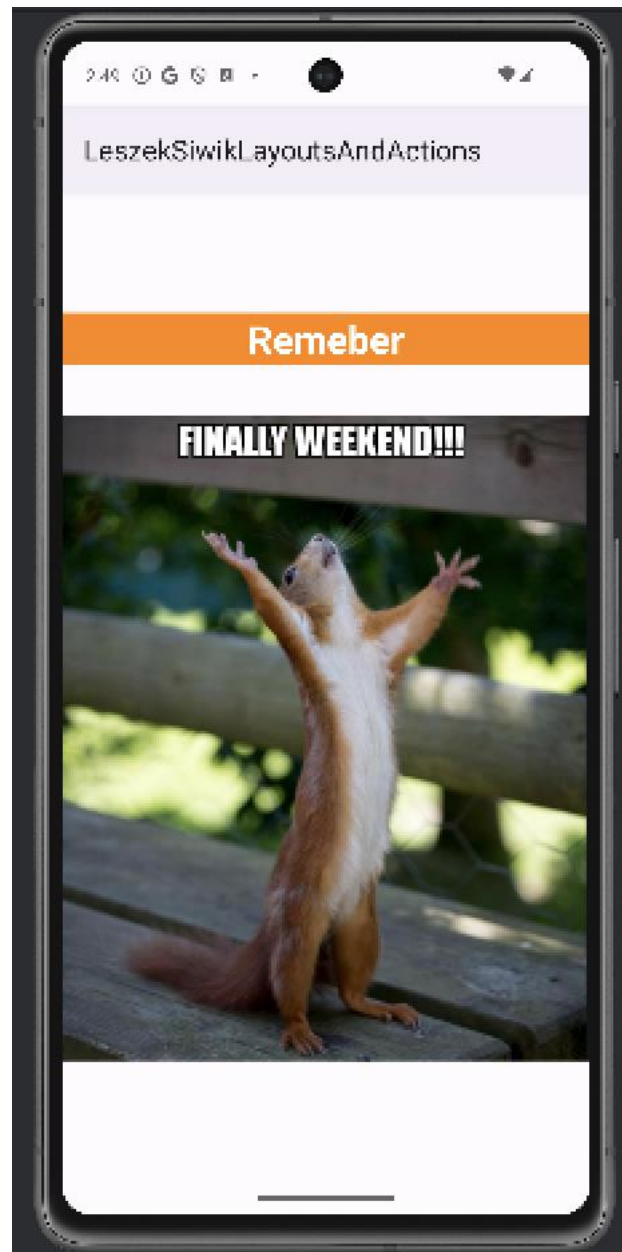
- which is our activity_main layout.
- This is because we didn't say the application to use the new layout. We have to do it in the MainActivity class:



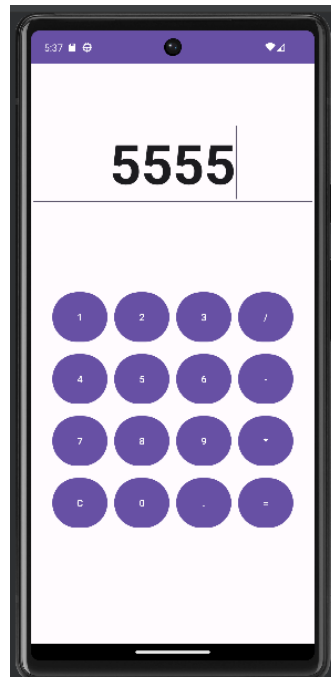
- Here in setContentView method we define which layout should be used – and we need to change it. Also we need to change the layout used for insets accordingly. And after that, it should work as expected, i.e.:



- For practicing – pls add the element marked in red below (so-called ActionBar) with the title of the application. To achieve that, we go to: res -> values -> themes -> themes.xml file and, in the Style section, we have to add two new items: i.e.: windowActionBar which should be set to true and windowNoTitle set to false.



- Now, please prepare by your own the following UI/layout definition



- When completed pls prepare the short pdf report containing code and app screenshots and upload it as of the Layouts task solution on UPEL platform